

UNIVERSIDAD DE SANTIAGO DE  
COMPOSTELA



ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA

# Optimización en el acceso a nubes de puntos de 3D mediante la utilización de octrees y coordenadas polares

*Autor/a:*

**Manuel Ramallo Blanco**

*Tutores:*

**Francisco M. Fernández Rivera**

**Miguel Yermo García**

**Grado en Ingeniería Informática**

**Junio 2026**

Trabajo de Fin de Grado presentado en la Escuela Técnica Superior de  
Ingeniería de la Universidad de Santiago de Compostela para la obtención del  
Grado en Ingeniería Informática





**D. (Nome da titora ou do titor)**, Profesor/a do Departamento de Electrónica e Computación da Universidade de Santiago de Compostela, e **D. (Nome da cotirora ou do cotitor)**, Profesor/a do Departamento de Electrónica e Computación da Universidade de Santiago de Compostela,

INFORMAN:

Que a presente memoria, titulada (*Título do traballo*), presentada por **D. (Nome da autora ou do autor do traballo)** para superar os créditos correspondentes ao Traballo de Fin de Grao da titulación de Grao en Enxeñaría Informática, realizouse baixo nosa tutoría no Departamento de Electrónica e Computación da Universidade de Santiago de Compostela.

E para que así conste aos efectos oportunos, expiden o presente informe en Santiago de Compostela, a (Data):

Titor/a,

Cotitor/a,

Alumno/a,

(Nome do titor/a) (Nome do cotitor/a) (Nome do alumn/a)



# Agradecimentos

Se se quere pór algún agradecemento, este vai aquí.



# Resumen

En este trabajo se describe y se presenta una solución de mejora de eficiencia temporal en el acceso a nubes de puntos 3D utilizando como estructura de datos *octrees* lineales. El estudio consiste en el reordenamiento espacial de puntos para tratar de explotar la capacidad de poda a la hora de encontrar vecindades. Se evalúan resultados temporales usando diferentes técnicas de reordenamiento, operando tanto con las coordenadas cartesianas como con las coordenadas polares de los puntos. Los resultados experimentales muestran un impacto positivo en la eficiencia temporal bajo determinadas circunstancias, a cambio de un gasto en memoria utilizada.





# Indice

|                                                             |           |
|-------------------------------------------------------------|-----------|
| <b>1. Introducción</b>                                      | <b>1</b>  |
| <b>2. Estado del arte del problema a abordar</b>            | <b>5</b>  |
| <b>3. Materiales</b>                                        | <b>9</b>  |
| 3.1. Equipos de cómputo y entorno de desarrollo . . . . .   | 9         |
| 3.2. Marco teórico matemático . . . . .                     | 10        |
| 3.2.1. Coordenadas tridimensionales . . . . .               | 10        |
| 3.2.2. Curvas de llenado de espacio . . . . .               | 12        |
| 3.3. Algoritmos y estructuras de datos de partida . . . . . | 13        |
| 3.3.1. Octree lineal . . . . .                              | 13        |
| 3.3.2. Algoritmos de búsqueda . . . . .                     | 14        |
| 3.4. Conjuntos de datos empleados . . . . .                 | 16        |
| <b>4. Metodología</b>                                       | <b>19</b> |
| <b>5. Pruebas</b>                                           | <b>21</b> |
| <b>6. Discusión de los resultados</b>                       | <b>23</b> |
| <b>7. Exemplos (eliminar capítulo na versión final)</b>     | <b>25</b> |
| 7.1. Un exemplo de sección . . . . .                        | 25        |
| 7.1.1. Un exemplo de subsección . . . . .                   | 25        |
| 7.1.2. Otro exemplo de subsección . . . . .                 | 25        |
| 7.2. Exemplos de figuras e cadros . . . . .                 | 26        |
| 7.3. Exemplos de referencias á bibliografía . . . . .       | 26        |
| 7.4. Exemplos de enumeracións . . . . .                     | 27        |
| <b>8. Conclusións e posibles ampliacións</b>                | <b>29</b> |
| <b>A. Manuais técnicos</b>                                  | <b>31</b> |
| <b>B. Manuais de usuario</b>                                | <b>33</b> |

|              |    |
|--------------|----|
| C. Licenza   | 35 |
| Bibliografia | 37 |

# Índice de figuras

|                                            |    |
|--------------------------------------------|----|
| 7.1. Esta é a figura de tal e cal. . . . . | 26 |
|--------------------------------------------|----|



# Índice de cuadros

|                                                                         |    |
|-------------------------------------------------------------------------|----|
| 3.1. Nubes de puntos utilizadas en las evaluaciones experimentales. . . | 17 |
| 7.1. Esta é a táboa de tal e cal. . . . .                               | 26 |



# Capítulo 1

## Introducción

Las nubes de puntos 3D son una representación de una realidad, como un objeto o un terreno, mediante un conjunto de puntos en el espacio tridimensional, estando cada punto caracterizado por sus coordenadas X, Y y Z así como otras propiedades como el color o la intensidad. Entre las aplicaciones más importantes de la elaboración de estas nubes están la cartografía, la elaboración de modelos tridimensionales en CAD de piezas fabricadas, la metrología, la arqueología o la conducción autónoma. La tecnología predominante para la captura de estos puntos es la LiDAR (Light Detection And Ranging).

Una de las tareas más comunes al trabajar sobre nubes de puntos 3D es la búsqueda de vecindades alrededor de un punto dado. Una vecindad se define como el conjunto de puntos de puntos que están dentro de una figura geométrica determinada en el espacio con radio  $R$  y con el centro en el punto para el cual se están buscando vecinos. Esta combinación de figura geométrica de y radio se conoce como kernel de búsqueda. Otro método alternativo de análisis de vecindades es buscar los  $K$  puntos más cercanos al elemento de partida, técnica llamada KNN (*k nearest neighbors*).

Un gran problema a nivel de hardware de este proceso sobre nubes de puntos es que estos datos se recogen sin ningún tipo de estructura ni orden basado en localidad. Este fenómeno se junta con el hecho de que estas nubes pueden tener millones de puntos almacenados. Por eso, se necesitan métodos eficientes para acelerar este lento proceso de acceso a puntos cercanos.

Los *Octrees* y los *KDTrees* son estructuras jerárquicas ampliamente utilizadas para lidiar con este problema de ineficiencia. Consisten en dividir la nube de puntos de manera recursiva en zonas próximas en el espacio físico. Los *Octrees* son estructuras donde cada nodo se divide exactamente 8 hijos, dividiendo cada dimensión a la mitad (solo se usa en espacios tridimensionales). Los puntos pertenecen al nodo hoja que los contiene en su interior. Por otra parte, los *KD-Trees* son árboles binarios que dividen cada nodo interno solamente según una dimensión. En cada nivel se elige un eje y se traza un hiperplano que divide a los puntos en dos nodos. De esta forma, este árbol es generalizable para cualquier

espacio de dimensiones.

El rendimiento de estas estructuras depende directamente de la distribución de los puntos. Por eso es muy útil la aplicación técnicas de reordenamiento espacial basadas en curvas de llenado de espacio (SFC). Las curvas de llenado son una representación mediante una línea continua de un cuadrado unidad bidimensional o, de forma general, un hipercubo  $N$  dimensional. En el caso de las nubes de puntos 3D nos interesa mapear los puntos tridimensionales a una representación unidimensional. El objetivo de la aplicación de curvas de llenado es mejorar la coherencia entre la localidad espacial de los puntos y su correspondiente localidad en memoria, con el fin de mejorar el rendimiento en esos accesos a memoria a la hora de calcular vecindades. Para lograr este reordenamiento, en la literatura destacan las curvas de Morton y las curvas de Hilbert, que implementan diferentes estrategias para proyectar los puntos en un índice lineal unidimensional.

En este trabajo se usará como base el modelo de *Octrees* propuesto por [1] donde se aplican las reordenaciones espaciales de Morton y Hilbert sobre diferentes implementaciones de *Octrees*, implementando tanto la estructura clásica basada en punteros como una versión lineal. Sobre esa implementación de algoritmo se tratará de optimizar todavía más el acceso a puntos vecinos. Para ello se empleará la técnica algorítmica de la poda.

Esta estrategia, de forma general, consiste en descartar o 'podar' regiones del espacio (nodos) o conjuntos de datos que matemáticamente se puede demostrar que no satisfacen las condiciones necesarias para ser solución. Su aplicación en este contexto es ser capaz de conocer *a priori* qué partes del *Octree* no pueden contener ningún vecino para una búsqueda dada. Esta técnica es, de hecho, el motivo por el que los *Octrees* (y los KDTrees) son estructuras ideales para almacenar nubes de puntos 3D, es muy sencillo podar nodos comprobando si sus límites están fuera del alcance del kernel de vecindad. Sin embargo, en este trabajo se explorará la poda a nivel de puntos dentro de un nodo, es decir, ser capaz de descartar puntos pertenecientes a un nodo hoja sin tener que evaluarlos individualmente. Para conseguir esta tarea se usarán reordenamientos de puntos a nivel de hoja siguiendo diversos criterios basados en coordenadas cartesianas con ejes X, Y, Z como definidores de los puntos, así como sistemas alternativos de de representaciones tridimensionales: sistemas esféricos y cilíndricos, que se tratan de expansiones de las coordenadas polares.

Las coordenadas polares son un sistema de coordenadas bidimensional donde cada punto es caracterizado por una distancia radial y un ángulo con respecto del origen. Las coordenadas cilíndricas añaden el componente de la altura a las propiedades de un punto. Por otro lado, las coordenadas esféricas descomponen la posición en el radio tridimensional, el ángulo  $\theta$  (coaltitud) y el ángulo  $\varphi$  (azimutal). En este contexto de nubes de puntos 3D almacenadas en *Octrees*, se tratará como el origen el centro geométrico de cada nodo.

Los reordenamientos en cuestión consisten en almacenar los puntos dentro de un nodo terminal de la estructura ordenados según alguna característica de las



mencionadas. Así, se podrán calcular los límites de forma teórica para ese valor que puede tener un punto de forma que satisfaga la restricción de vecindad, dada una búsqueda con un kernel determinado.

Se estudiará el impacto en materia de eficiencia temporal de la aplicación de esta poda teórica de puntos en la implementación del *Octree* lineal propuesta en [1] con respecto a los algoritmos usados inicialmente. Finalmente, también se analizará la escalabilidad de la técnica, paralelizando la búsqueda de vecinos.



## Capítulo 2

# Estado del arte del problema a abordar

El uso de Octrees como estructura de datos en la gestión y almacenamiento de nubes de puntos 3D es en la actualidad predominante a nivel global. Existen librerías de software libre ampliamente reconocidas que proporcionan soporte nativo tanto a estructuras Octrees como a KD-Trees para optimizar la indexación espacial.

Una de estas librerías es Open3D. Está escrita en C++ pero cuenta con interfaces estables para numerosos lenguajes, considerándose de facto el estándar en Python para la manipulación y visualización 3D. Open3D incluye métodos avanzados para el cálculo de distancias y la búsqueda de vecindad, dando soporte tanto a consultas basadas en kernels de radio como a la técnica de los  $K$  vecinos más cercanos (*K-Nearest Neighbors* o KNN).

Otro gigante en el manejo de nubes de puntos, considerada la librería de referencia absoluta en el entorno académico e industrial para el procesamiento de geometría 3D en C++, es PCL (*Point Cloud Library*) [ref pcl]. PCL proporciona un ecosistema masivo y modular que da soporte nativo a estructuras espaciales mediante las clases `pcl::Octree` y `pcl::KdTree`. El enfoque de PCL en sus Octrees tradicionales se basa en una estructura dinámica de punteros. Este diseño otorga una gran versatilidad para realizar modificaciones en tiempo real sobre la nube, como la inserción o eliminación dinámica de puntos y la detección de cambios espaciales. Por la contra, introduce una severa penalización en el rendimiento de la memoria. Esto se debe a la incoherencia entre la localidad espacial de los datos y su disposición física en el *heap* o montón, provocando constantes fallos de caché y saltos de punteros en el direccionamiento de la CPU durante los recorridos de búsqueda.

Un aspecto crítico que comparten tanto Open3D como PCL es que no implementan sus algoritmos de búsqueda desde cero, sino que integran internamente la librería FLANN (*Fast Library for Approximate Nearest Neighbors*) [ref flann]. FLANN es una biblioteca altamente optimizada para la búsqueda de vecinos más

cercanos en espacios vectoriales multidimensionales. Si bien su principal característica es la capacidad de realizar búsquedas aproximadas (*Approximate Nearest Neighbor* o ANN) para mitigar la maldición de la dimensionalidad sacrificando un margen mínimo de precisión, en el contexto de búsquedas exactas por radio o kernel (como las evaluadas en este estudio) se configura en su modo exacto para operar como línea base de comparación directa.

Otras librerías relevantes con el mismo propósito, escritas también en C++, son PicoTree y NanoTree. PicoTree genera un KD-Tree integrado en un array contiguo mediante la indexación previa de los puntos, permitiendo búsquedas esféricas y de KNN altamente eficientes. Por su parte, NanoTree destaca por ser una implementación ligera y compacta optimizada para sistemas con restricciones de recursos.

Por otro lado, las curvas de llenado del espacio (SFC) son ampliamente aplicadas a estructuras jerárquicas para tareas de indexado o compresión. La integración de códigos de Morton en estructuras de Octree lineales ha sido una estrategia clave para mejorar la eficiencia en la ordenación de puntos. Un ejemplo de ello es el trabajo de Behley et al. con la librería *unibn* [2], donde se emplean estos códigos para reducir la necesidad de recorrer el árbol en toda su profundidad durante las búsquedas de vecindad, logrando superar el rendimiento de otras herramientas populares de la disciplina. Estos códigos son reconocidos por su simplicidad algorítmica y rapidez de cómputo a nivel de bits. Sin embargo, el otro gran algoritmo de SFC, las curvas de Hilbert, ha ganado una notable popularidad en la literatura reciente por garantizar una preservación superior de la localidad espacial, asegurando que si dos puntos son cercanos en el espacio tridimensional, mantendrán dicha proximidad en la proyección unidimensional de memoria de forma estricta.

No obstante, la aplicación directa de SFC en la búsqueda de vecindades en nubes de puntos 3D obtenidas con sensores LiDAR fue implementada originalmente en el trabajo de Díaz de la Iglesia et al. [1]. En dicha investigación se analiza en primer lugar una estructura Octree de punteros estándar construida mediante fragmentaciones recursivas. A pesar de que a esta estructura se le pueden aplicar reordenamientos según curvas de Morton o Hilbert para mejorar la cercanía en memoria de los puntos próximos, el enfoque clásico sigue adoleciendo del problema de los accesos indexados: cada transición entre nodos durante el recorrido implica saltar a una dirección del mapa de memoria que puede estar totalmente dispersa.

Por este motivo, los autores propusieron e implementaron una versión lineal del Octree, la cual almacena todos los puntos de la nube en un único vector contiguo, garantizando que todos los elementos pertenecientes a un mismo nodo hoja residan en posiciones físicas consecutivas del array. Adicionalmente, se introdujo un algoritmo de búsqueda por kernel geométrico que optimiza el proceso de recorrido: a diferencia del método clásico que desciende recursivamente verificando la mera intersección, este algoritmo identifica de forma temprana si el volumen

del kernel contiene por completo el área que cubre el nodo (sea este interno o terminal). De cumplirse esta condición de inclusión total, el algoritmo realiza un volcado secuencial directo de todos los puntos del subárbol, abortando el descenso y acelerando drásticamente la consulta.

En la publicación de Díaz de la Iglesia et al. [1] se presentan resultados experimentales que demuestran la superioridad del Octree lineal (en sus variantes con reordenamiento de Morton y Hilbert) frente al Octree clásico de punteros. Asimismo, se incluye un exhaustivo análisis comparativo de rendimiento frente a las soluciones de código abierto de referencia mencionadas anteriormente, tales como los módulos de PCL, PicoTree, NanoTree y la librería *unibn*. Bajo diferentes tipologías de datasets y variaciones en el radio del kernel de búsqueda, el Octree lineal reportó, de manera sistemática, los menores tiempos de ejecución computacional.

Sobre este escenario técnico se fundamenta el presente trabajo. En la práctica totalidad de las librerías del estado del arte existe un patrón de diseño común: las estructuras justifican su eficiencia basándose exclusivamente en la aplicación de la poda a nivel de nodos intermedios (ramas). Mediante funciones de solapamiento geométrico, se descartan de forma temprana subárboles completos cuyos volúmenes no intersecan con el kernel de consulta. Sin embargo, una vez que el algoritmo alcanza un nodo hoja válido, se ve obligado a evaluar la distancia de todos los puntos contenidos en él de manera individual y secuencial.

Hasta donde se tiene constancia en la literatura técnica y en las soluciones de código abierto analizadas, no existe ninguna implementación que acote el rango de puntos a evaluar mediante una poda matemática interna que actúe de forma nativa **en el interior del propio nodo hoja**.

Este cuello de botella constituye la principal motivación y el núcleo de la contribución de este trabajo. Al introducir funciones de cálculo de rango basadas en reordenamientos geométricos locales (tanto en sistemas cartesianos como esféricos y cilíndricos), este estudio permite aislar zonas y subrangos de puntos dentro de los nodos hoja. Esto cambia la concepción que se tenía sobre los nodos hoja, anteriormente pensados como unidades atómicas e indivisibles. Este enfoque persigue un incremento en el rendimiento temporal de la estructura del Octree lineal, motivada por la reducción del número de puntos evaluados como candidatos a vecinos. Como un extra, también busca aportar una mayor flexibilidad al diseño del software, permitiendo configurar tamaños de hoja más elevados sin penalizar el tiempo de cómputo por desbordamiento de búsquedas exhaustivas.



# Capítulo 3

## Materiales

En este capítulo se definirán y precisarán todas las herramientas, métodos, algoritmos y equipos que fueron empleados y tomados para la realización del estudio.

### 3.1. Equipos de cómputo y entorno de desarrollo

El entorno hardware en el que fueron desarrollados los experimentos de este proyecto es el Centro de Supercomputación de Galicia (CESGA). Este centro es un nodo de la Red Española de Supercomputación y datos (RES) y cuenta con una capacidad de computación total de 4.36 PetaFLOPS, estando compuesto por 357 nodos interconectados a través de una red Infiniband HDR.

Existen diferentes grupos de nodos con distintas especificaciones hardware. En concreto, estos experimentos fueron realizados en un nodo del grupo grupo COMPLETAAAAR.

El entorno de software configurado y desplegado sobre esta arquitectura de hardware consta de las siguientes especificaciones:

- **Sistema Operativo:** Distribución Rocky Linux versión 8.4 (*Green Obsidian*), compatible con entornos Red Hat Enterprise Linux (RHEL).
- **Estándar de Lenguaje:** Se empleó el estándar **C++20**.
- **Compilador:** Las implementaciones fueron compiladas mediante la *GNU Compiler Collection* (GCC), cargando explícitamente el módulo `gcc/12.3.0`.
- **Directivas de Optimización y Extensiones Vectoriales:** Para la generación de los binarios ejecutables, se aplicó el nivel de optimización alta `-O2` junto con la macro `-DNDEBUG` para inhabilitar aserciones de depuración y maximizar el rendimiento. Adicionalmente, el sistema realiza un análisis de

compatibilidad por hardware en el nodo de cómputo para activar de forma nativa las siguientes extensiones vectoriales:

- `-mavx2` (*Advanced Vector Extensions 2*): Permite la vectorización de bucles mediante instrucciones SIMD en registros de 256 bits, procesando múltiples coordenadas de puntos 3D de forma simultánea.
- `-mbmi2` (*Bit Manipulation Instruction Set 2*): Introduce soporte directo por hardware para instrucciones avanzadas de manipulación de bits (como PDEP y PEXT).

Asimismo, se habilitó la optimización interprocedural o en tiempo de enlazado (LTO/IPO) a través de CMake, forzando a GCC a optimizar el proyecto de forma global y permitiendo la inserción en línea (*inlining*) de funciones entre diferentes archivos fuente.

- **Soporte de Paralelismo:** Se incluyó la API **OpenMP versión 4.5** encargada de la gestión y paralelización de hilos en memoria compartida. Se aplica en el código añadiendo directivas que definen el nivel de concurrencia en la inicialización de los bucles de búsqueda masiva sobre la estructura del Octree.

Aunque el descrito fue el entorno de ejecución real, se utilizó un entorno local de desarrollo para la programación y depuración del código fuente. Este entorno físico de soporte está constituido por una arquitectura de cómputo x86\_64 basada en un procesador AMD Ryzen 7 6800H de 8 núcleos físicos y 16 hilos de ejecución concurrentes y un sistema de almacenamiento de memoria caché multinivel (L1, L2 y L3 de 512 KiB, 4 MiB y 16 MiB respectivamente).

Con el fin de compatibilizar el ecosistema de herramientas de desarrollo y mantener un entorno de terminal nativo unificado, el software se implementó sobre una capa de virtualización completa mediante el subsistema *Windows Subsystem for Linux* (WSL2) bajo el hipervisor de Microsoft. Sobre esta capa se desplegó una distribución del sistema operativo Linux, concretamente Ubuntu 24.04.3 LTS. Este entorno local sirvió como plataforma de validación para el diseño de los algoritmos, la verificación de la correcta aplicación de la poda y su integración con el código original antes de su posterior despliegue y compilación en los nodos de cómputo del CESGA.

## 3.2. Marco teórico matemático

### 3.2.1. Coordenadas tridimensionales

La base de este trabajo es optimizar el cálculo de distancias entre dos puntos en un espacio tridimensional. Concretamente, se trabaja con dos figuras geométricas (cubo y esfera) que envuelven los puntos de búsqueda. En ambas se hace uso



de las 3 componentes cartesianas de un punto tridimensional en las comprobaciones de vecindad:

- **Kernel esférico:** Dos puntos son vecinos si la distancia  $d_{ij}$  entre el centro de búsqueda es menor al radio de búsqueda.

- Fórmula de la distancia:

$$d_{ij} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2 + (z_i - z_j)^2}$$

- Existe vecindad si se cumple:  $d_{ij} < R_{\text{búsqueda}}$

- **Kernel cúbico:** Dos puntos son vecinos si la distancia entre los puntos para los tres ejes dimensionales por separado es menor al radio del cubo de búsqueda:

- Para un eje  $k \in \{X, Y, Z\}$ , la distancia  $d_{kij}$  se define como:

$$d_{kij} = |k_i - k_j|$$

- Existe vecindad si se cumple:

$$|x_i - x_j| \leq R \wedge |y_i - y_j| \leq R \wedge |z_i - z_j| \leq R$$

Para potenciar la poda o el descarte de puntos sin tener que realizar esta comprobación, se hará uso de otros sistemas de coordenadas tridimensionales existentes aparte de los cartesianos: el sistema esférico y el cilíndrico.

Ambos son extensiones del sistema bidimensional polar. En estas coordenadas un punto se define a través de dos magnitudes: un radio  $R$  y un ángulo  $\theta$  sobre el origen. Para la proyección a 3 dimensiones, existe un sistema que añade un ángulo con respecto al plano y proyecta el valor del radio bidimensional a un radio tridimensional (coordenadas esféricas). Por otra parte, se puede incluir la componente  $Z$ , que le otorga la propiedad de altura a los puntos (coordenadas cilíndricas). La conversión de un punto a estos sistemas a partir de sus componentes se hace a través de las siguientes operaciones:

- **Coordenadas esféricas**

$$R = \sqrt{x^2 + y^2 + z^2}$$

$$\varphi = \arctan(y/x)$$

$$\theta = \arccos(z/R)$$

- **Coordenadas cilíndricas**

$$R = \sqrt{x^2 + y^2}$$

$$\varphi = \arctan(y/x)$$

$$Z = z$$

En el contexto concreto de los puntos dentro del Octree, se debe tener en cuenta que para poder operar a nivel de hoja se debe trasladar el origen de las coordenadas al centro del octree antes de realizar cualquier conversión u operación.

### 3.2.2. Curvas de llenado de espacio

Con el fin de optimizar la localidad espacial de los datos en memoria se introducen las curvas de llenado del espacio (SFC, *Space-Filling Curves*) como herramientas de ordenación. Se encargan de forzar que aquellos puntos próximos en el espacio tridimensional se almacenen de forma contigua en la zona de memoria reservada para la nube.

El proceso requiere la discretización previa del espacio continuo de la nube de puntos. Sea una nube de puntos  $P \subset \mathbb{R}^3$ , se define su caja contenedora o *bounding box* como la región semiabierta  $B = [x_m, x_M) \times [y_m, y_M) \times [z_m, z_M)$ , delimitada por sus esquinas inferiores y superiores. Fijando un nivel de subdivisión  $L \in \mathbb{N}$  que determina una resolución de  $N = 2^L$ , se proyecta una rejilla discreta tridimensional  $S_L = [0, N) \times [0, N) \times [0, N) \subset \mathbb{N}^3$ , cuya cardinalidad total es de  $2^{3L}$  celdas. Cada punto continuo  $p = (x, y, z) \in P$  es trasladado, escalado y transformado en un elemento entero de la rejilla  $S_L$  mediante la aplicación de la función suelo.

En la implementación práctica de este estudio se fija un nivel máximo de subdivisión  $L = 21$ . Esto permite codificar la posición tridimensional de cada celda en un único identificador entero de 64 bits (restringiendo las coordenadas discretas originales a tipos de 32 bits), minimizando la probabilidad de colisión de múltiples puntos en una misma celda sin penalizar el rendimiento del sistema. Una curva de llenado del espacio de nivel  $l$  se define formalmente como la función biyectiva:

$$C : S_l \longrightarrow [0, 2^{3l})$$

Para el propósito de evaluar el impacto de la localidad en el rendimiento de los algoritmos de búsqueda, se consideran dos familias de curvas de llenado del espacio recursivas:

#### Código de Morton (Curva Z)

La curva de Morton, o curva Z, mapea las coordenadas tridimensionales discretas mediante el entrelazado directo de sus bits a nivel de registros de la CPU. Para un punto discretizado en la rejilla cuyas coordenadas binarias se expresan como  $p = (x_0 \dots x_{L-1}, y_0 \dots y_{L-1}, z_0 \dots z_{L-1}) \in S_L$ , su código de Morton se calcula de acuerdo con la ecuación:

$$C_M(x_0 \dots x_{L-1}, y_0 \dots y_{L-1}, z_0 \dots z_{L-1}) = x_0 y_0 z_0 x_1 y_1 z_1 \dots x_{L-1} y_{L-1} z_{L-1}$$

La ventaja computacional de esta estructura radica en la velocidad y eficiencia de su cálculo. Al tratarse de un mapeo posicional directo de bits, la codificación y decodificación puede implementarse a través de operaciones lógicas de enmascaramiento (*bitmasks*) y tablas de búsqueda (LUT, *Look-Up Tables*).

### Curva de Hilbert

La curva de Hilbert es una curva de llenado continua que elimina los saltos espaciales característicos de la curva de Morton. El cálculo del código de Hilbert se realiza mediante un proceso iterativo por nivel de subdivisión en el cual, tras extraer los bits homólogos de cada dimensión, se aplica una secuencia de transformaciones geométricas y rotaciones que dependen del estado de orientación previo y del octante evaluado.

Matemáticamente, la propiedad fundamental de las curvas de Hilbert es su continuidad en el espacio, satisfaciendo la condición de vecindad de forma que dos códigos consecutivos en el espacio unidimensional corresponden a celdas contiguas en el espacio tridimensional:

$$\|C^{-1}(i) - C^{-1}(i - 1)\|_{\infty} = 1, \quad \forall i \in [1, 2^{3l-1}] \quad (3.1)$$

A nivel teórico, esta curva ofrece una localidad espacial superior a la de Morton. Sin embargo, carece de una correspondencia de bits directa y requiere un algoritmo iterativo más costoso para computar las matrices de rotación. Sin embargo, debe prevalecer la calidad sobre la complejidad a la hora de calcular los reordenamientos de las nubes de puntos, ya que es una operación que solo hace falta realizar una vez. Se puede ejecutar incluso en un programa separado y cargar las codificaciones cuando se inicialice la nube.

## 3.3. Algoritmos y estructuras de datos de partida

En este apartado se describirán las estructuras y explicarán los algoritmos de búsqueda que se usaron como base para el trabajo y que ya fueron mencionados: el Octree lineal implementado por [1] y sus algoritmos de búsqueda iterativos con poda a nivel de nodo.

### 3.3.1. Octree lineal

El proceso de construcción es un proceso iterativo, en el cual se parte de un nodo raíz que contiene toda la *bounding box* de la nube y divide repetidamente cada nodo terminal que contenga más de *maxLeaf* puntos, siendo este un valor paramétrico introducido por el programador. Una vez finalizada la construcción

se obtiene un número  $N$  de nodos, separados en  $nInternal$  (número de nodos internos) y  $nExternal$  (número de nodos hoja). Todo el Octree está formado por el vector de puntos de la nube (reordenados mediante una curva de llenado y por lo tanto con una codificación asociada) junto con una serie de vectores persistentes que mapean las relaciones de descendencia entre nodos sin necesidad de usar punteros. Esta serie de vectores es la siguiente:

- `std::vector<uint32_t>offsets`

Tiene  $N$  elementos. La posición  $i$  almacena el índice de del primer hijo del nodo  $i$ . Todos los hijos de un nodo están en posiciones consecutivas. En caso de ser el nodo  $i$  un nodo hoja se cumple que `offsets[i] == 0`.

- `std::vector<std::pair<size_t, size_t>>internalRanges`

Tiene  $N$  elementos. En cada índice guarda las posiciones inicial y final de los puntos contenidos en él. Se usa para recorrer los puntos uno a uno en caso de llegar a una hoja que interseca con el kernel. Este es el punto de ineficiencia sobre el que se despliega este proyecto.

- `std::vector<Point>centers`

Cada elemento `centers[i]` contiene las coordenadas del centro geométrico del nodo  $i$ . También tiene  $N$  elementos.

- `std::vector<Vector>precomputedRadii`

Tiene  $L$  elementos siendo  $L$  la profundidad máxima encontrada en el Octree. Almacena para cada nivel de profundidad las dimensiones de los octantes.

Este formato presenta una serie de ventajas con respecto a un Octree de punteros. Se eliminan las indirecciones que provocan fallos caché y se asegura que toda la estructura esté en un grupo de bloques de memoria contiguos. Además, se conoce y se tiene acceso a los puntos contenidos por un octante, ya sea una hoja o no, y se consigue que esos puntos estén consecutivos en memoria gracias a la reordenación por SFC.

### 3.3.2. Algoritmos de búsqueda

A diferencia de las aproximaciones clásicas basadas en punteros, donde el recorrido del árbol requiere desreferenciar enlaces hacia nodos hijos dispersos en el montón o *heap*, el algoritmo optimizado implementado en este proyecto explota la geometría de los octantes y la contigüidad de los índices para discriminar bloques enteros de puntos.

El procedimiento general de búsqueda geométrica se rige por un esquema de recorrido recursivo sobre los nodos del árbol, evaluando la colisión espacial entre la caja del nodo actual y el volumen del kernel de consulta. El flujo algorítmico se define en los siguientes pasos:

1. **Evaluación de nodos tipo rama:** Si el nodo actual del octree es una rama (nodo interno), se computa la posición relativa de su volumen geométrico respecto a las fronteras del kernel de búsqueda:
  - a) **Inclusión completa:** Si el octante se encuentra totalmente contenido en el interior del kernel, se activa el mecanismo de rangos internos (*internal ranges*). Esto permite indexar y volcar de manera secuencial todos los puntos contenidos en dicho subárbol sin necesidad de descender más niveles ni evaluar individualmente sus elementos.
  - b) **Exclusión completa:** Si el octante se halla completamente fuera de los límites del kernel, se procede a la poda (*pruning*) inmediata de la rama, abortando esa vía de exploración.
  - c) **Intersección parcial:** Si el octante interseca con las fronteras del kernel pero no está contenido estrictamente en él, se continúa el recorrido recursivo descendiendo hacia sus nodos hijos.
2. **Evaluación de nodos tipo hoja:** Al alcanzar un nodo hoja que interseca parcialmente con el kernel, el algoritmo desciende al nivel atómico del dato, validando punto a punto mediante criterios geométricos si sus coordenadas pertenecen o no al interior del volumen de consulta.
3. **Retorno de resultados:** Tras concluir el recorrido, se consolida y devuelve la colección de índices de los puntos que se encuentran estrictamente contenidos en el espacio delimitado por el kernel.

Para la materialización y almacenamiento de estos índices resultantes, se han desarrollado dos variantes algorítmicas diferenciadas por su estrategia de gestión de memoria y estructuración de datos: `neighborsPrune` y `neighborsStruct`.

#### Algoritmo `neighborsPrune`

Esta variante implementa una estrategia de inserción directa y plana. Cuando el algoritmo identifica puntos válidos en el interior del kernel (ya sea mediante la evaluación individual en hojas o por la inclusión completa de una rama), añade cada índice de punto vecino como un elemento independiente y separado dentro de un vector dinámico estándar (`std::vector<size_t>`). Si bien el coste de inserción es secuencial, esta aproximación puede resultar innecesaria en nubes densas, donde se insertan numerosos puntos consecutivos.

#### Algoritmo `neighborsStruct`

Con el objetivo de mitigar el coste de inserción masiva de elementos individuales, la variante `neighborsStruct` aprovecha la contigüidad física de los datos reordenados en memoria. En lugar de almacenar elementos aislados, define una estructura de rangos indexados mediante el tipo:

```
std::vector<std::pair<size_t, size_t>> ranges;
```

Bajo este esquema, cuando una secuencia de puntos consecutivos en memoria pertenece de forma unánime al kernel de consulta (caso seguro al incluir ramas totalmente contenidas), el algoritmo compacta la información realizando únicamente dos operaciones de inserción por bloque: el índice de la posición inicial y el índice de la posición final del rango. Esta técnica reduce drásticamente el espacio de almacenamiento temporal, minimiza las escrituras en memoria caché, optimizando la velocidad del algoritmo.

### 3.4. Conjuntos de datos empleados

Las nubes de puntos 3D que fueron usadas para evaluar el rendimiento del algoritmo de poda a nivel de hoja vienen descritos a continuación, en el Cuadro 3.1. Las nubes están almacenadas en archivos binarios estándar con extensión **.LAS**. Este formato contiene los datos espaciales tridimensionales aparte de campos opcionales con más propiedades de los puntos, como intensidad, color o clasificaciones de terreno.

La selección de nubes de puntos se realizó tratando de buscar la máxima variedad posible: se mezclan conjuntos de datos de baja y alta densidad, así como nubes extraídas con sensores LiDAR tanto aéreos como terrestres y por último variedad en el número total de puntos contenidos.

| Nube                                        | nPuntos | Dataset        | Descripción                                                                                                               |
|---------------------------------------------|---------|----------------|---------------------------------------------------------------------------------------------------------------------------|
| <i>bildstein_station1_xyz_intensity_rgb</i> | 29.6 M  | Semantic3D     | Escaneo LiDAR terrestre y urbano desde una posición estática. Nubes muy densas.                                           |
| <i>sg27_intensity_rgb</i>                   | 73.9 M  |                |                                                                                                                           |
| <i>5085_54320.las</i>                       | 14.4 M  | DALES          | LiDAR aéreo obtenido desde aeronaves. El conjunto se divide en celdas disjuntas.                                          |
| <i>5095_54440.las</i>                       | 11.7 M  |                |                                                                                                                           |
| <i>Lille_0</i>                              | 10.0 M  | Paris-Lille 3D | Sensor LiDAR de tipo Mobile Laser Scanning (MLS) instalado sobre un vehículo que mapea dos calles urbanas en París.       |
| <i>Paris-Luxembourg_6</i>                   | 10.0 M  |                |                                                                                                                           |
| <i>PNOA_2024_PNR_489-4672_NPC01</i>         | 18.5 M  | PNOA-II        | Cobertura del territorio español mediante LiDAR aéreo. Presenta una densidad baja, cubriendo grandes extensiones.         |
| <i>Mar18_train</i>                          | 59.4 M  | Hessigheim 3D  | Dataset aéreo de alta resolución capturado mediante UAV ( <i>Unmanned Aerial Vehicle</i> ) sobre el pueblo de Hessigheim. |
| <i>Mar18_test</i>                           | 51.7 M  |                |                                                                                                                           |

Cuadro 3.1: Nubes de puntos utilizadas en las evaluaciones experimentales.





# Capítulo 4

## Metodología

Ah shit here we go...



# Capítulo 5

## Pruebas



## Capítulo 6

# Discusión de los resultados

Comprobación da(s) hipótese(s) de partida do traballo. Comparativa con algunha das solucións similares presentadas no estado de coñecemento, resaltando por que o traballo presentado mellora as solucións anteriores.



# Capítulo 7

## Exemplos (eliminar capítulo na versión final)

### 7.1. Un exemplo de sección

Esta é *letra cursiva*, esta é **letra negrilla**, esta é letra subrallada, e esta é **letra curier**. Letra tiny, scriptsize, small, large, Large, LARGE e moitas más. Exemplo de fórmula:  $a = \int_0^\infty f(t)dt$ . E agora unha ecuación aparte:

$$S = \sum_{i=0}^{N-1} a_i^2. \quad (7.1)$$

As ecuaciones se poden referenciar: ecuación (7.1).

#### 7.1.1. Un exemplo de subsección

O texto vai aquí.

#### 7.1.2. Otro exemplo de subsección

O texto vai aquí.

#### Un exemplo de subsubsección

O texto vai aquí.

#### Un exemplo de subsubsección

O texto vai aquí.

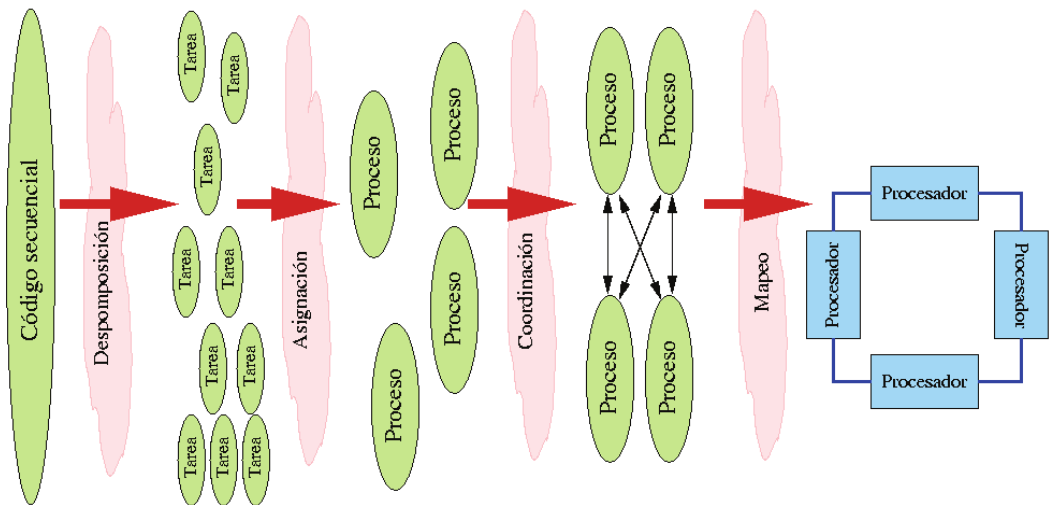


Figura 7.1: Esta é a figura de tal e cal.

| Izquierda | Derecha | Centrado |
|-----------|---------|----------|
| ll        | r       | cccc     |
| llll      | rrr     | c        |

Cuadro 7.1: Esta é a táboa de tal e cal.

Un exemplo de subsubsección

O texto vai aquí.

7.2. Exemplos de figuras e cadros

A figura número 7.1.  
O cadro (taboa) número 7.1.

7.3. Exemplos de referencias á bibliografía

Este é un exemplo de referencia a un documento descargado da web [1]. E este é un exemplo de referencia a unha páxina da wikipedia [2]. Agora un libro [3] e agora unha referencia a un artigo dunha revista [4]. Tamén se poden por varias referencias á vez [1, 3].



## 7.4. Ejemplos de enumeraciones

Con puntos:

- Un.
- Dous.
- Tres.

Con números:

1. Catro.
2. Cinco.
3. Seis.

Exemplo de texto verbatim:

```
0 texto          verbatim
  se visualiza tal
    como se escribe
```

Exemplo de código C:

```
#include <math.h>
main()
{  int i, j, a[10];
   for(i=0;i<=10;i++) a[i]=i; // comentario 1
   if(a[1]==0) j=1; /* comentario 2 */
   else j=2;
}
```

Exemplo de código Java:

```
class HelloWorldApp {
    public static void main(String[] args) {
        System.out.println("Hello World!"); // Display the string.
    }
}
```



## Capítulo 8

# Conclusións e posibles ampliacións

Resumo das principais aportacións do traballo, explicacións das suposicións e limitacións do traballo, e posibles vías de mellora.



# Apéndice A

## Manuais técnicos

En función do tipo de Traballo e metodoloxía empregada, o contido poderase dividir en varios documentos. En todo caso, neles incluírase toda a información precisa para aquelas persoas que desexen repetir o experimento (por exemplo código fonte, recursos necesarios, operacións necesarias para modificacións e probas, posibles problemas, etc.). O código fonte poderase entregar en soporte informático en formatos PDF ou postscript.



## Apéndice B

### Manuais de usuario

Incluirán toda a información precisa para aquelas persoas que repliquen o experimento: instalación, utilización, configuración, mensaxes de erro, etc. A documentación do usuario debe ser autocontida, é dicir, para o seu entendemento o usuario final non debe precisar da lectura doutro manual técnico.





# Apéndice C

## Licenza

Se se quere pór unha licenza (GNU GPL, Creative Commons, etc), o texto da licenza vai aquí.



# Bibliografía

- [1] Nvidia CUDA programming guide. Versión 2.0, 2010. Disponible en <http://www.nvidia.com>.
- [2] Acceso múltiple por división de código. Artigo da wikipedia (<http://es.wikipedia.org>). Consultado o 2 de xaneiro do 2010.
- [3] R.C. Gonzalez e R.E. Woods, *Digital image processing*, 3ª edición, Prentice Hall, New York, 2007.
- [4] P. González, J.C. Cartex e T.F. Pelas, “Parallel computation of wavelet transforms using the lifting scheme”, *Journal of Supercomputing*, vol. 18, no. 4, pp. 141-152, junio 2001.